

Advanced Big Data : Hadoop

1. Introduction Big Data

* **Définition** : Le Big Data désigne des ensembles de données issus de **multiples sources hétérogènes** constituant **une très grande quantité** de données difficilement traitable avec les outils classiques de gestion de base de données même en utilisant les machines les plus puissantes.

Quand peut-on dire qu'on travaille avec du big data ?

* **Caractéristiques Big data (3V)**

Volume : Des données massives

Variété : Des données de plusieurs sources sous plusieurs formes (modèles de données) :

=> **Données structurées** (schéma prédéfini) : BDR

=> **Données semi-structurées** (schéma défini ultérieurement) : xml, fichier log, pdf...

=> **Données non structurées** (pas de schéma) : image, vidéo, blog...

Vélocité : La vitesse avec laquelle les données sont générées, analysées et traitées (en temps réel)

[5V]

Valeur : La transformation des données en des informations utiles sur le plan stratégique, économique de l'entreprise...

Véracité : La qualité + la fiabilité des données

* **Cas d'utilisation :**

=> Marketing prédictif (des prévisions basées sur les données des utilisateurs), sport, détection de fraudes...

2. Hadoop

=> Framework qui permet le stockage distribué (HDFS) et le traitement parallèle du Big Data (MapReduce)

* **Caractéristiques**

=> Hadoop n'est pas conçu pour des requêtes temps réel ou de faible latence.

=> Hadoop est performant dans le traitement des batch hors ligne d'un grand volume de données.

=> Les SGBDNR sont meilleures pour les transactions en ligne et de faible latence.

* **Atouts de Hadoop**

La gestion des défaillances : que ce soit au niveau du stockage ou traitement, les nœuds responsables de ces opérations durant le processus de Hadoop sont automatiquement gérés en cas de défaillance.

La sécurité et persistance des données : il n'y a plus de soucis de perte de données.

La montée en charge : garantie d'une montée en charge maximale.

La complexité réduite : capacité d'analyse et de traitement des données à grande échelle.

Le coût réduit : Hadoop est open source, et malgré leur massivité et complexité, les données sont traitées efficacement et à très faible coût

* Fonctionnement

=> Diviser les données

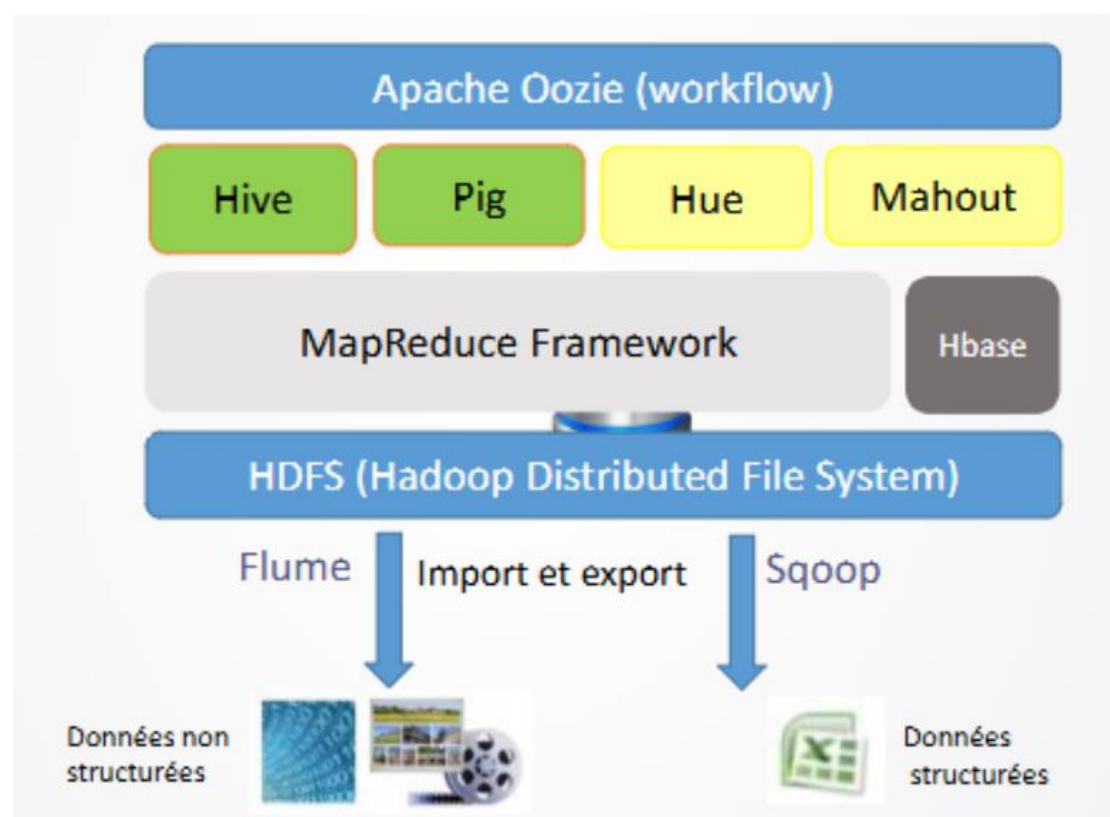
=> Les sauvegarder sur un ensemble de machines, appelé cluster

=> Traiter les données directement là où elles sont stockées

Il est possible d'ajouter des machines au cluster, au fur et à mesure que les données augmentent => **Scalabilité horizontale

Scalabilité  **Scalabilité horizontale** (ajouter des machines)
Scalabilité verticale (ajouter des ressources à une machine : RAM, CPU....) => Limitée

* Ecosystème Hadoop



Composants de base :

HDFS : Hadoop Distributed FileSystem

MapReduce : framework de traitement des données distribuées

=> En plus de HDFS et MapReduce, on trouve plusieurs autres composants :

Oozie : Planificateur et manager du workflow Hadoop

HBase : base de données Hadoop orientée colonne

Pig : Langage de traitement de données

Hive : Data warehouse avec interface SQL

Hue : Front-end graphique pour le cluster, représente un navigateur pour HDFS et HBase. Des éditeurs pour Hive, Pig, Impala et Sqoop.

Mahout : Bibliothèque d'apprentissage automatique.

=> Outils de transfert de données :

Sqoop : Prend les données à partir d'une base de données relationnelle, et les met dans HDFS, comme étant des fichiers délimités, pour être traitées avec d'autres données dans le cluster

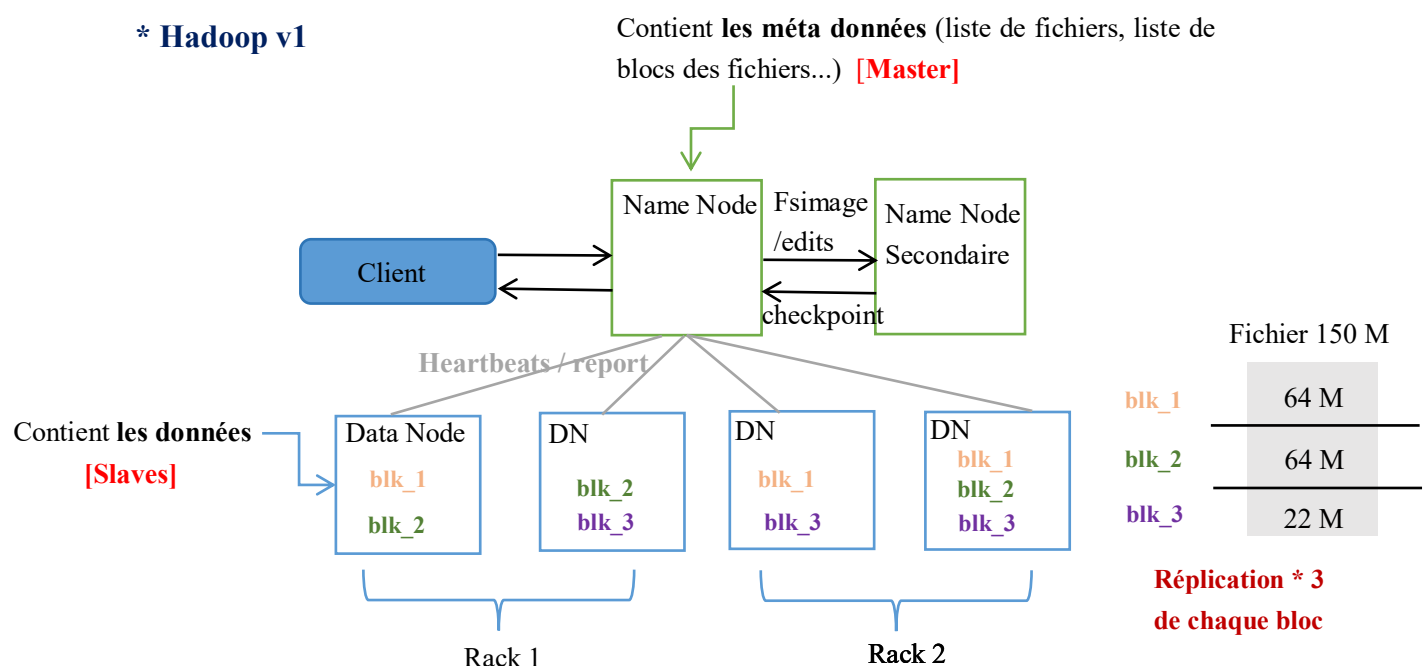
Flume : Système distribué permettant de collecter, regrouper et déplacer efficacement un ensemble de données non structurées à partir de plusieurs sources vers HDFS.

* Distributions Hadoop

- ✓ Cloudera Distribution for Hadoop (CDH)
- ✓ MapR Distribution
- ✓ Hortonworks Data Platform (HDP)
- ✓ IBM InfoSphere BigInsights

3. HDFS (Hadoop Distributed FileSystem)

* Hadoop v1



* Comment se fait le stockage des données sous hdfs ?

On suppose qu'on a un fichier de taille 150 M

=> On ne peut pas stocker le fichier dans un seul nœud car s'il tombe en panne, les données seront perdues donc :

=> La première étape consiste à découper ce fichier en **des blocs** de taille **64 M** (version 1 de hadoop), on aura donc 3 blocs (64M 64M 22M)

=> Hadoop **réplique** chaque bloc **3 fois**

=> Il choisit 3 nœuds et place une copie du bloc dans chacun d'eux (on place les blocs dans des nœuds différents et rack différent pour avoir toujours accès aux données)

=> Si le nœud est en panne, le NameNode le détecte et s'occupe de répliquer encore les blocs qui y étaient hébergés pour avoir toujours 3 copies.

* Rôle du NameNode

=> C'est le master

=> Il contient les **méta-données** (liste des fichiers, liste des blocs pour chaque fichier...)

=> Il contient **4 fichiers** :

* **Version** : contient des informations sur le namenode (id, date...)

* **Edits** : regroupe toutes les opérations effectuées dans le système (backup)

* **Fsimage** : fichier binaire utilisé comme un point de contrôle sur les méta-données (backup)

* **Fstime** : stocke les informations sur la dernière opération de contrôle sur les données

* Rôle du NameNode Secondaire

=> Il lit les méta données à partir du RAM du NameNode et les écrit sur le disque dur

=> Il vérifie périodiquement l'état du NN primaire, il copie le fichier Edits et applique les modifications au fichier fsimage

* Problèmes du NameNode

=> Si le namenode est en panne, l'administrateur doit le restaurer **manuellement** en utilisant le NameNode secondaire (il va utiliser le fichier fsimage)

=> Les données seront temporairement inaccessibles

=> Le NameNode est un **SPOF** (Single Point of Failure)

=> Haute disponibilité inexistante !!!

=> Le NameNode secondaire n'est pas un fail-over pour le NN, ne garantit pas la haute disponibilité et n'améliore pas les performances du NN

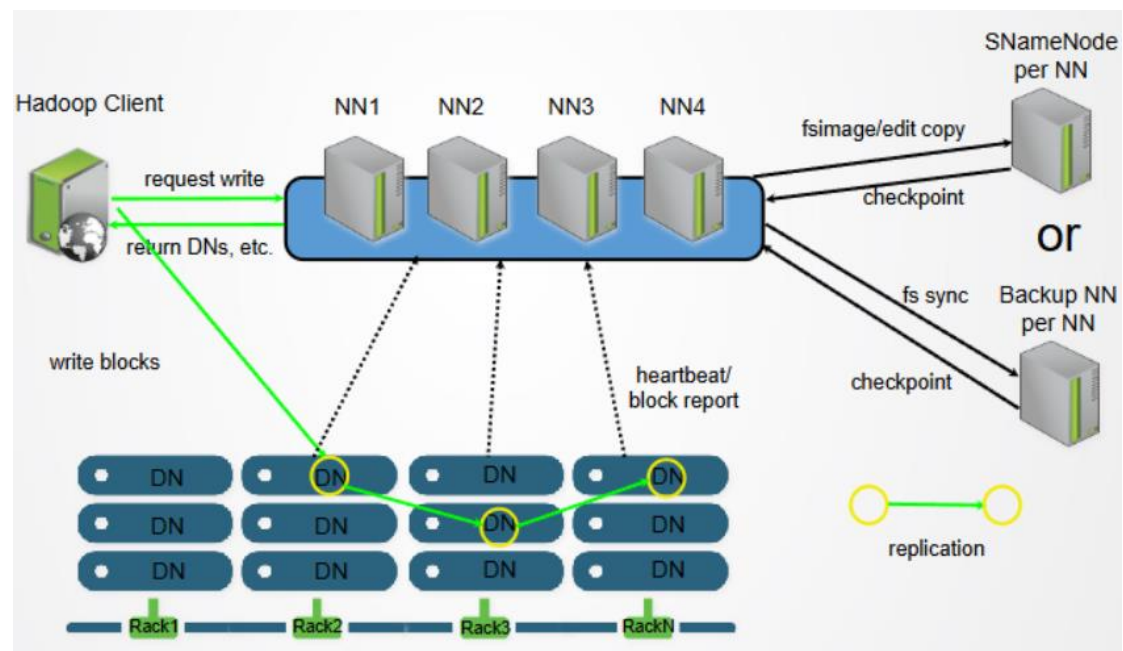
* Hadoop v2

=> On a une Scalabilité horizontale du NameNode (backup)

=> NN n'est plus un SPOF (on a une version active et une version passive du NN)

=> Les NN sont indépendants, tous les fichiers Edits sont stockés dans un système de fichier partagé, le NN passif lit à partir de ce fichier, si l'actif tombe en panne le NN

passif le détecte et commence à écrire dans l'espace partagé



Propriétés	HDFS 1.0	HDFS 2.0
Taille de Bloc	64 Mo	128 Mo
Intervention Manuelle	Obligatoire	Non requise
Haute disponibilité	N'est pas assurée (SPOF)	Assurée (Standby NN)
Fédération	Un seul namenode	Plusieurs namenode

* Commandes Shell

hadoop fs - <command> -<option> <URI>
ou
hdfs dfs

Répertoire Local : /home/cloudera

Répertoire sous HDFS : /user/cloudera

=> **Afficher le contenu d'un document (25 premières lignes)**

`hadoop fs -cat fichier.txt | head -n 25`

=> **Copier le fichier de HDFS vers HDFS**

`hadoop fs -cp fichier.txt fichier1.txt`

=> **Copier le fichier du LOCAL vers HDFS**

`hadoop fs -put fichier.txt fichier1.txt`

On peut aussi utiliser copyFromLocal

=> **Copier des fichier à partir du HDFS**

`hadoop fs -get fichier.txt fichier1.txt`

On peut aussi utiliser copyToLocal

=> **Afficher le contenu d'un répertoire**

hadoop fs -ls rép

=> **Créer un répertoire**

hadoop fs -mkdir rép

=> **Supprimer un fichier**

hadoop fs -rm fichier

=> **Supprimer un répertoire**

hadoop fs -rmdir rép (il faut qu'il soit vide)

hadoop fs -rm -r rép (forcer la suppression)

=> **Afficher la taille d'un fichier**

hadoop fs -du fichier

hadoop fs -du -h fichier (lisible par l'être humain)

hadoop fs -du -s rép (afficher la taille d'un répertoire)

=> **Contrôle des incohérences/ problèmes (blocs manquants...)**

hadoop fsck /

!! ne corrige pas les problèmes (c'est le rôle du NameNode)

=> **Statistiques du HDFS**

hadoop dfsadmin -report

=> **Facteur de réplication**

hadoop fs -ls fichier

=> **Nombre de blocs**

hadoop fsck fichier

4. MapReduce

=> Patron d'architecture de développement

=> Permet de traiter des données volumineuses de manière **parallèle** et **distribuée**

=> Au lieu de parcourir le fichier séquentiellement, il est divisé en blocs qui sont parcourus en //

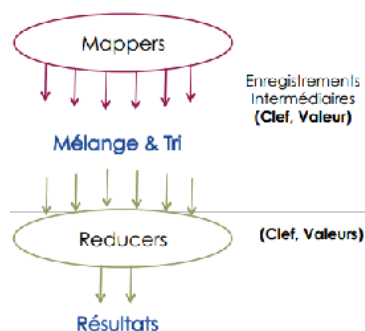
=> Il se base sur 3 étapes :

*Mapping : une fonction dont le but est d'analyser des données brutes

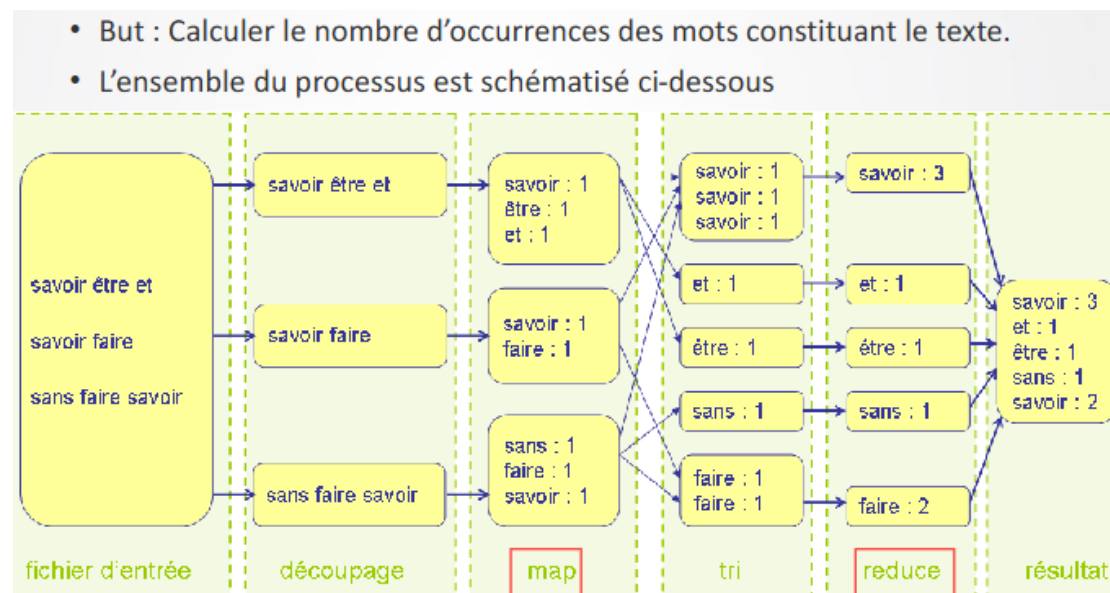
Output : clé-valeur

*Mélange et Tri

*Reducing : cette tâche récupère les données construites dans l'étape du mappage pour en extraire des informations pertinentes

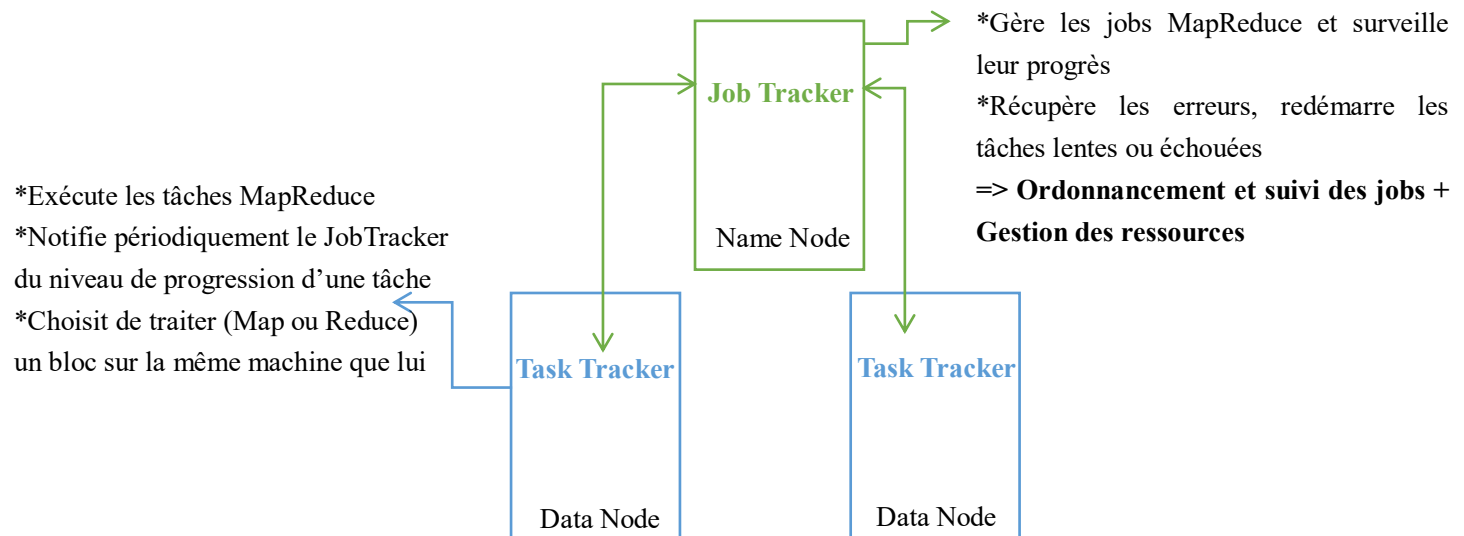


Exemple :



* Architecture MapReduce v1

On a deux processus : **JobTracker** et **TaskTracker**



=> Comment se fait **la gestion des ressources** ?

MapReduce v1 a un modèle de gestion de ressources qui est inflexible, statique basé sur **les slots (unité de stockage)**

Chaque TaskTracker est configuré au démarrage pour avoir un nb bien déterminé de slots (map slots/reduce slots) pour l'exécution des tâches

Dans un slot, on peut faire que map ou reduce mais pas les deux à la fois

* Limites de MapReduce v1

=> **Scalabilité limitée** : Le JobTracker s'exécute sur une seule machine et a deux

rôles : Ordonnancement et suivi des jobs + Gestion des ressources

=> **Problème de disponibilité** : Le JobTracker est un SPOF, s'il est endommagé tous les jobs doivent être redémarrés

=> **Problème d'utilisation des ressources** : Il y a un nombre prédéfini de map slots et reduce slots pour chaque TaskTracker

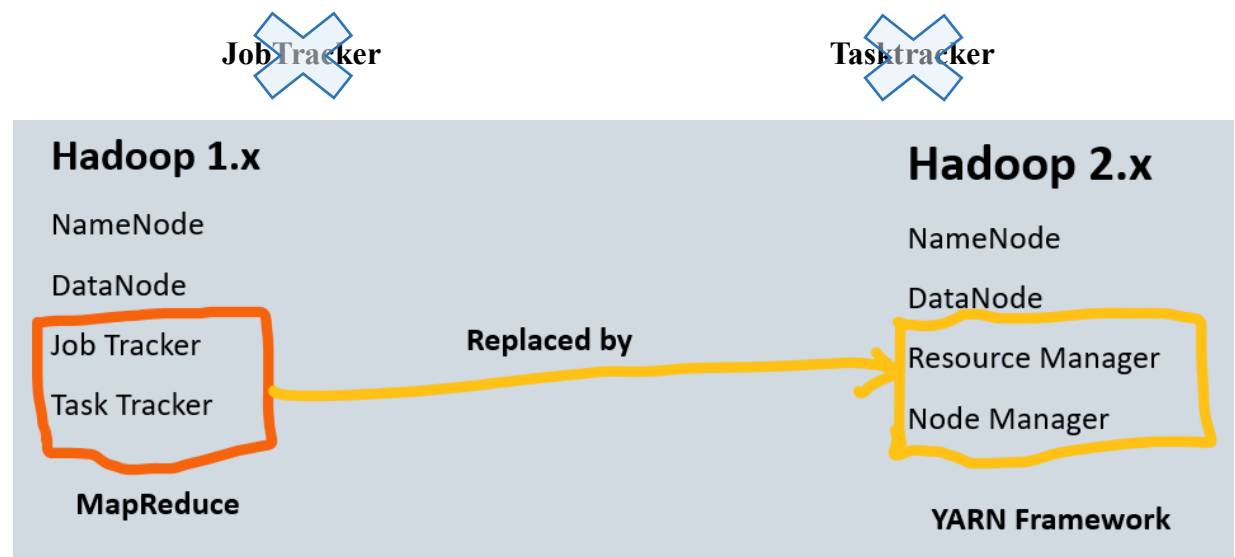
=> **Utilisation des applications Non-Mapreduce** : Le JobTracker est intégré à MapReduce et ne supporte que les applications MapReduce (si on veut appliquer une opération il faut passer le mapping et le reducing)

La solution est :

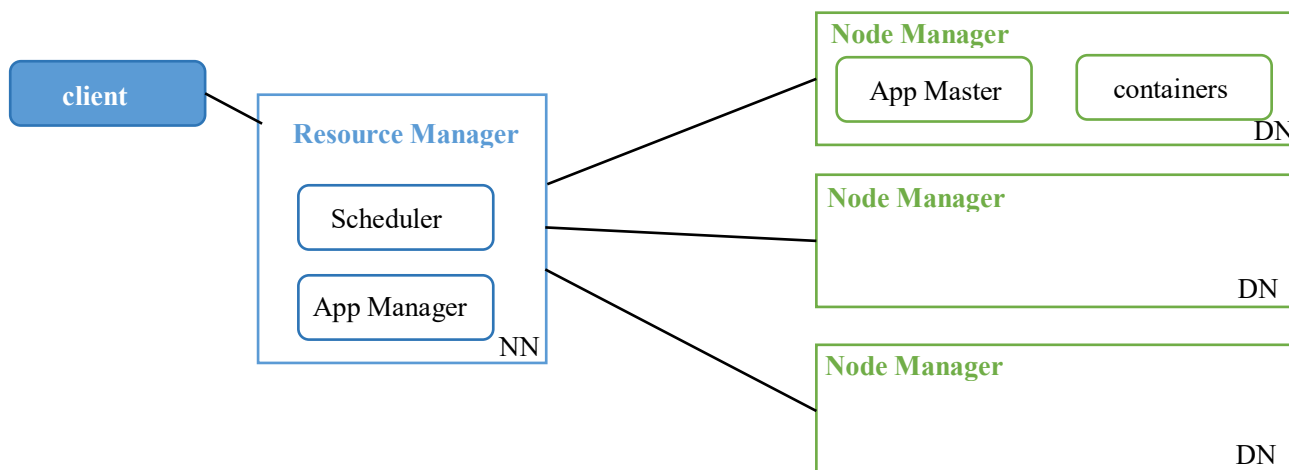
* MapReduce v2

=> **YARN** (Yet Another Resource Negotiator) : Utilisation des applications non MapReduce + Séparation des deux fonctionnalités du JobTracker : Ordonnancement et suivi des jobs + Gestion des ressources **en 2 composants séparés** :

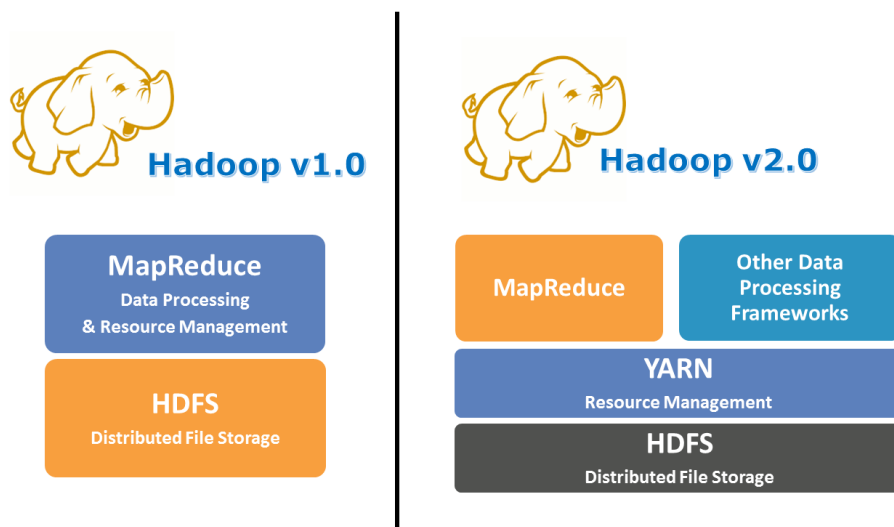
Gestionnaire des ressources (Resource Manager) + **Gestionnaire de l'application** (Node Manager)



* Architecture MapReduce v1



- => On a un client qui va lancer une application
- => Le Resource Manager va être notifié, il va déclencher le Node Manager et plus précisément l'Application Master du NM
- => L'App Master va évaluer le traitement à exécuter puis va demander les ressources nécessaires pour l'exécution de ces tâches (**Gestion dynamique**) de la part du RM
- => Le RM va allouer les ressources nécessaires c'est à dire les **containers nécessaires** et il va envoyer les IDs de ces containers à l'App Master
- => L'App Master va déclencher l'exécution de la tâche dans ces containers



5. Hive

- => Solution Data Warehouse de hadoop
- => Fournit un langage de requête similaire au SQL nommé : HiveQL
- => Traduit les requêtes HiveQL en **un ensemble de jobs MapReduce** qui seront exécutés dans un cluster hadoop

* Les commandes Hive

- => **Démarrage Hive** : `hive`
- => **Création d'une base de données** : `> create database test`
`> use test`
- => **Création d'une table** :

```
CREATE TABLE posts (user STRING, post STRING, time BIGINT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS
TEXTFILE ;
```

(Pour le TERMINATED BY ',' il faut voir le délimiteur du fichier qu'on va charger la table avec)
- => **Insertion de données dans une table** :

```
LOAD DATA LOCAL INPATH 'path to the file' OVERWRITE INTO TABLE posts;
```

- => **Liste des tables** : show tables ;
- => **Description de la table** : describe posts;
- => **Affichage des données** : select * from posts ;
- => **Suppression d'une table** : DROP TABLE posts;

Répertoire de HIVE : /user/hive/warehouse

Si on crée une table nommée posts, un dossier nommé posts va être créé sous le répertoire warehouse, si on ouvre ce répertoire on voit le fichier qu'on a chargé la table avec.

[Home](#)
/ [user](#) / [hive](#) / [warehouse](#) / **posts**
▼ History

☐	Name	Size	User	Group	Permissions	Date
☐	↑		hive	supergroup	drwxrwxrwx	December 25, 2024 11:00 AM
☐	.		cloudera	supergroup	drwxrwxrwx	December 25, 2024 11:03 AM
☐	posts.txt	42 bytes	cloudera	supergroup	-rwxrwxrwx	December 25, 2024 11:03 AM

Si on supprime la table posts, le dossier posts va être supprimé et donc le fichier posts.txt (suppression des données et méta-données)

=> EXTERNAL table

* Création :

CREATE EXTERNAL TABLE post (user STRING, post STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE LOCATION '/user/cloudera/test'

=> Hive charge les fichiers dans le répertoire /user/cloudera/test et non pas dans le datawarehouse Hive (le fichier post.txt va être sous le répertoire test)

[Home](#)
/ [user](#) / **cloudera**
▼ History

☐	Name	Size	User	Group	Permissions	Date
☐	↑		hdfs	supergroup	drwxr-xr-x	July 19, 2017 06:31 AM
☐	.		cloudera	cloudera	drwxr-xr-x	December 25, 2024 11:10 AM
☐	test		cloudera	cloudera	drwxr-xr-x	December 25, 2024 11:10 AM

Si on supprime la table post, les méta données de la table seront supprimées et non pas les données qu'on a chargé la table avec.

=> Partitions

* Création :

CREATE TABLE posts (user STRING, post STRING, time BIGINT)

PARTITIONED BY (country STRING)

ROW FORMAT DELIMITED FIELDS TERMINATED BY ',' STORED AS TEXTFILE;

* Chargement de données dans des tables partitionnées:

LOAD DATA LOCAL INPATH '/home/cloudera/user-posts-US.txt' OVERWRITE INTO TABLE posts **PARTITION (country = 'US');**

LOAD DATA LOCAL INPATH '/home/cloudera/user-posts-TUN.txt' OVERWRITE INTO TABLE posts **PARTITION (country = 'TUN');**

=> Les partitions sont physiquement stockées dans des **répertoires distincts**

=> **Pour afficher les partitions :**

hive> show partitions posts;

ou

Hadoop fs -ls -R /user/hive/warehouse/posts

[Home](#) / [user](#) / [hive](#) / [warehouse](#) / [posts](#) ▼ History

☐	Name	Size	User	Group	Permissions	Date
☐	↑		hive	supergroup	drwxrwxrwx	December 25, 2024 11:20 AM
☐	.		cloudera	supergroup	drwxrwxrwx	December 25, 2024 11:22 AM
☐	country=TUN		cloudera	supergroup	drwxrwxrwx	December 25, 2024 11:22 AM
☐	country=US		cloudera	supergroup	drwxrwxrwx	December 25, 2024 11:21 AM

Sous chaque répertoire, on trouve le fichier qu'on a chargé la partition avec.

```
[cloudera@quickstart ~]$ hadoop fs -ls -R /user/hive/warehouse/posts
drwxrwxrwx - cloudera supergroup      0 2024-12-25 11:24 /user/hive/warehouse/posts/country=TUN
-rwxrwxrwx 1 cloudera supergroup     19 2024-12-25 11:24 /user/hive/warehouse/posts/country=TUN/posts-TUN.txt
drwxrwxrwx - cloudera supergroup      0 2024-12-25 11:24 /user/hive/warehouse/posts/country=US
-rwxrwxrwx 1 cloudera supergroup     20 2024-12-25 11:24 /user/hive/warehouse/posts/country=US/posts-US.txt
```

=> Jointure

* On va faire une jointure entre la table likes et la table posts

* On va tout d'abord créer la table posts_likes

CREATE TABLE posts_likes (user STRING,post STRING,likes_count INT);

* On va maintenant la charger en utilisant une jointure

INSERT OVERWRITE TABLE posts_likes SELECT p.user,p.post,l.nbllike FROM posts p JOIN likes l ON (p.user=l.user);

=> **Cette requête va lancer un job MapReduce**

6. Sqoop

Sqoop est idéal pour les tâches d'import/export de données entre bases de données relationnelles et Hadoop

* Importation des données sans méta données

```
sqoop import-all-tables --connect jdbc:mysql://localhost:3306/retail_db --username=root --password=cloudera --warehouse-dir=/user/hive/warehouse/h_retail_db.db --m=1
```

On va se connecter à MySQL pour accéder à la base de données retail_db en utilisant root comme username et cloudera comme mot de passe, on va importer toutes les tables sous cette base de données et les mettre sous le répertoire de la base de données h_retail_db créée sous hive. On va importer les données sans méta données (schéma, types de colonnes...) et on va utiliser un seul mapper (--m 1). On doit ensuite créer les tables sous hive et pointer la table vers les fichiers importés qui contiennent les données.

```
Create external table h_departments(department_id int, department_name string)
row format delimited fields terminated by ','
Location '/user/hive/warehouse/h_retail_db.db/departments';
select * from h_departments;
```

* Importation des données avec méta données (AVRO files)

```
sqoop import-all-tables \
-m 1 \
--connect jdbc:mysql://localhost:3306/retail_db \
--username=root --password=cloudera \
--compression-codec=snappy \
--as-avrodatafile \
--warehouse-dir=/user/hive/warehouse/retail_avro.db
```

On va refaire la même chose mais cette fois on va importer les données avec les méta données (schéma, types de colonnes....) sous format avro (**--as-avrodatafile**)

En créant les tables, on va pointer sur les fichiers qui contiennent les données (.avro) et ceux qui contiennent le schéma (.avsc)

Les fichiers .avsc sont importés dans le local (/home/cloudera), on doit les déplacer sous hdfs

```
CREATE EXTERNAL TABLE ord_item STORED AS avro
LOCATION '/user/hive/warehouse/retail_avro.db/order_items'
TBLPROPERTIES ('avro.schema.url' = 'hdfs:///user/cloudera/retail_schema/order_items.avsc');
```

* Importations des données à travers une sous-interrogation directement sous Hive

```
sqoop import --connect jdbc:mysql://localhost:3306/retail_db --username=root --password=cloudera --hive-import --table=categories --where "category_id between 1 and 22" --m=1
```

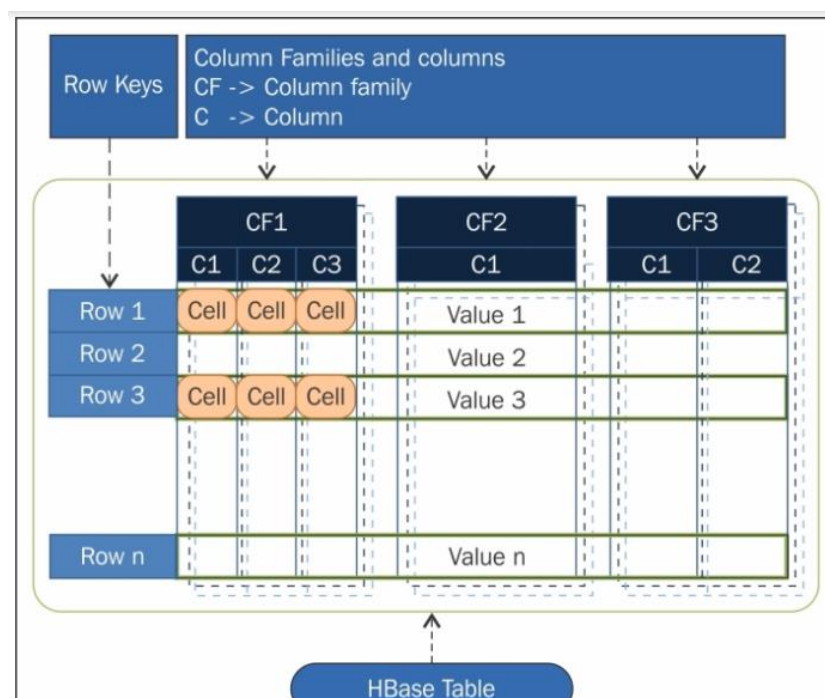
On va importer la table categories avec la condition : category id entre 1 et 22

7. Hbase

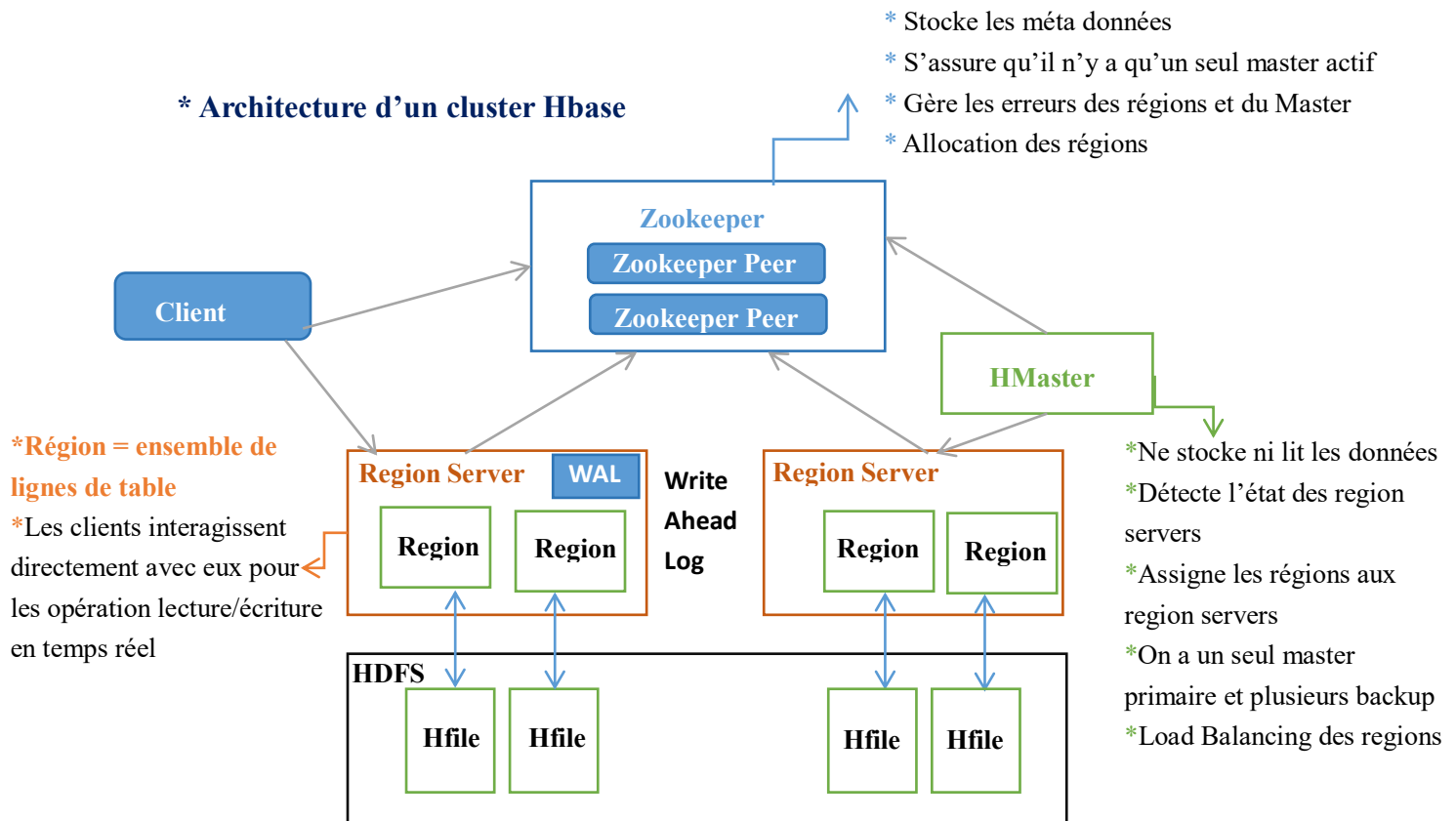
- => Base de données **orientée colonnes**, nommée Hadoop Database
- => Supporte des opérations CRUD **en temps réel**
- => Distribuée, conçue pour stocker des tables de grand volume
- => Scalabilité horizontale (shard automatique), forte cohérence de données, intégration avec le framework MapReduce

* Modèle de données Hbase

- => Les données sont stockées dans des tables
- => Les tables contiennent des lignes référencées par une clé
- => Les lignes sont composées de colonnes groupées par **famille**
- => Les données sont stockées dans **des cellules**
- => Une cellule est identifiée par une combinaison : (rowkey,famille de colonne,colonne,timestamp)
- => Pour chaque cellule, plusieurs versions sont sauvegardées (dans hadoop v1 3 sont sauvegardées par défaut, dans hadoop v2 une seule est sauvegardée par défaut)
- => Les versions sont stockées dans un ordre décroissant du timestamp (type long)
- => On peut spécifier le nombre de versions qui doivent être gardées



* Architecture d'un cluster Hbase



=> On utilise le Hfile pour le stockage des données contenus dans les régions

=> On a une seule famille de colonne par Hfile

=> La taille maximale d'un Hfile est 64M si on travaille avec hadoop v1, 128M si on travaille avec hadoop v2

=> On va décomposer chaque hfile en blocs et chaque bloc est répliqué au minimum 3 fois pour garantir la haute disponibilité

▪ Table - - - - - - Contains column-families ▪ Column family - - - - - - Logical and physical grouping of columns ▪ Column - - - - - - Exists only when inserted - Can have multiple versions - Each row can have different set of columns - Each column identified by it's key ▪ Row key - - - - - - Implicit primary key - Used for storing ordered rows - Efficient queries using row key	<table border="1"> <thead> <tr> <th colspan="2">HBTABLE</th> </tr> <tr> <th>Row key</th> <th>Value</th> </tr> </thead> <tbody> <tr> <td>11111</td> <td> cf_data: { 'cq_name': 'name1', 'cq_val': 1111 } cf_info: { 'cq_desc': 'desc11111' } </td> </tr> <tr> <td>22222</td> <td> cf_data: { 'cq_name': 'name2', 'cq_val': 2013 @ ts = 2013, 'cq_val': 2012 @ ts = 2012 } </td> </tr> </tbody> </table>	HBTABLE		Row key	Value	11111	cf_data: { 'cq_name': 'name1', 'cq_val': 1111 } cf_info: { 'cq_desc': 'desc11111' }	22222	cf_data: { 'cq_name': 'name2', 'cq_val': 2013 @ ts = 2013, 'cq_val': 2012 @ ts = 2012 }
HBTABLE									
Row key	Value								
11111	cf_data: { 'cq_name': 'name1', 'cq_val': 1111 } cf_info: { 'cq_desc': 'desc11111' }								
22222	cf_data: { 'cq_name': 'name2', 'cq_val': 2013 @ ts = 2013, 'cq_val': 2012 @ ts = 2012 }								

HFile

```
11111 cf_data cq_name name1 @ ts1
11111 cf_data cq_val 1111 @ ts1
22222 cf_data cq_name name2 @ ts1
22222 cf_data cq_val 2013 @ ts1
22222 cf_data cq_val 2012 @ ts2
```

HFile

```
11111 cf_info cq_desc desc11111 @ ts1
```

* Hbase shell

=> **Démarrage Hbase** : hbase shell

=> **Création de table** : **create** 't1',{NAME=>'fc1'},{NAME=>'fc2'}

On définit les familles de colonnes lors de la création de la table

On peut définir le nombre de versions par famille de colonne

create 'users',{NAME=>'infos',VERSIONS=>3},{NAME=>'adresse'}

=> **Insertion / Mise à jour des données** : **put** 't1','row_id','famille:colonne','valeur'

put 'users','user-1','infos:name','jihene'

***Ajout** de la ligne si elle n'existe pas

***Mise à jour** si la ligne existe (ajouter une nouvelle version pour la cellule)

=> **Affichage des données** :

***Afficher une seule ligne** : **get** 'users','user-1' (select * from users where id='user-1')

***Affichage des colonnes bien déterminées** :

get 'users','user-1',{COLUMN=>'infos:nom'} [une seule colonne]

get 'users','user-1',{COLUMN=>['infos:nom','adresse:rue']} [plusieurs colonnes]

***Affichage pour un timestamp bien déterminé** :

get 'users','user-1',{COLUMN=>'infos:nom',TIMESTAMP=>1234666}

***Affichage de plus qu'une version** :

get 'users','user-1',{COLUMN=>'infos:nom',VERSIONS=>2}

***Affichage de toutes les lignes d'une table** :

scan 'users' (select * from users)

* **Limiter le nombre de lignes affichées** :

scan 'users',{LIMIT=>2}

* **Affichage d'un ensemble de données** :

Scan 'users',{COLUMN=>'infos:nom',STARTROW=>'user-1',STOPROW=>'user-3'}

=> **Suppression des données** :

***Suppression des cellules** : **delete** 'users','user-1','infos:nom' (supprimer toutes les versions d'une ligne)

On doit supprimer colonne par colonne comme dans l'insertion des données

***On peut ajouter le timestamp pour supprimer une version spécifique** :

On suppose qu'on a ces 3 versions pour la colonne nom :

```
hbase(main):006:0> scan 'users',{COLUMN=>'infos:nom',VERSIONS=>3}
ROW          COLUMN+CELL
user-2       column=infos:nom, timestamp=1735165588477, value=jihene
user-2       column=infos:nom, timestamp=1735165577976, value=aziz
user-2       column=infos:nom, timestamp=1734012089144, value=mustpha
1 row(s) in 0.0140 seconds
```

Si on veut supprimer 'jihene', il faut **ajouter 1 au timestamp** car Hbase supprime toutes les versions antérieures du timestamp donné :

delete 'users','user-2','infos:nom', 1735165588478

(Si on donne le timesmtap exact de jihene, les 3 versions vont être supprimées)

Si on veut supprimer mustpha on peut donner le timestamp exact sans ajouter 1 car mustpha est la version la plus ancienne (il n'a pas des versions antérieures)

```
hbase(main):009:0> delete 'users','user-2','infos:nom',1734012089144
0 row(s) in 0.0070 seconds
```

```
hbase(main):010:0> scan 'users',{COLUMN=>'infos:nom',VERSIONS=>3}
ROW          COLUMN+CELL
user-2       column=infos:nom, timestamp=1735165588477, value=jihene
user-2       column=infos:nom, timestamp=1735165577976, value=aziz
1 row(s) in 0.0230 seconds
```

*Suppression de la table :

On doit d'abord désactiver la table avant la suppression

1) **disable** 'users'

2) **drop** 'users'

Pour afficher la liste des tables : list

Pour afficher le nombre de lignes : count 'users'

count 'users', {INTERVAL=>2}

```
hbase(main):015:0> scan 'users'
ROW          COLUMN+CELL
user-1       column=infos:age, timestamp=1734010630156, value=22
user-1       column=infos:prenom, timestamp=1734012069202, value=hajji
user-2       column=infos:age, timestamp=1734012097443, value=23
user-2       column=infos:nom, timestamp=1735166133171, value=amira
user-2       column=infos:prenom, timestamp=1734012082922, value=aloui
user-3       column=infos:nom, timestamp=1735166143509, value=amira
user-4       column=infos:nom, timestamp=1735166152702, value=ahmed
4 row(s) in 0.0460 seconds
```

```
hbase(main):016:0> count 'users',{INTERVAL=>2}
Current count: 2, row: user-2
Current count: 4, row: user-4
4 row(s) in 0.0380 seconds
```